

# Linux IPTables Firewall Project Report

---

Name: Mubeen Khan

Project: IPTables Firewall Configuration & Testing

Date: 16 July 2025

Role: Security Operations (SOC) Analyst – Student Simulation (CYBRARY)

## 1. Introduction

This project involves configuring and testing a Linux firewall using the iptables utility. The purpose of the project is to simulate a real-world SOC (Security Operations Center) task where specific IP addresses are allowed access to certain services while denying others. The project is divided into two main parts: configuring firewall rules (Part 1) and testing them in a simulated environment (Part 2). This hands-on project demonstrates my ability to manage host-based firewalls, apply security policies, and validate configurations using network tools.

## 2. Part 1 – Configuring IPTables Firewall

To simulate a realistic testing environment for our iptables firewall, we created a fictional internal network scenario. In this setup, the remote lab system functions as both a web server and an FTP server. Within this network, there are two client machines: one designated as the **admin** with the IP address 10.0.0.1, and the other as the **user** with the IP address 10.0.0.2.

The security policy for this network is as follows:

- Only the **admin (10.0.0.1)** should be allowed to access the system via **SSH (port 22)**
- All devices within the **10.0.0.0/24** subnet should be allowed **HTTP (port 80)** access to the web server
- Only the **user (10.0.0.2)** should be granted **FTP (port 21)** access

Based on these access control requirements, a corresponding iptables ruleset was developed to enforce them accordingly.

| Task   | Chain | Source      | Protocol | Port | Rule   | Meaning                                    |
|--------|-------|-------------|----------|------|--------|--------------------------------------------|
| APPEND | INPUT | 10.0.0.1    | TCP      | 22   | ACCEPT | Allow 10.0.0.1 SSH access                  |
| APPEND | INPUT | 10.0.0.0/24 | TCP      | 22   | DROP   | Deny the 10.0.0.0 network SSH access       |
| APPEND | INPUT | 10.0.0.2    | TCP      | 21   | ACCEPT | Allow 10.0.0.2 FTP access                  |
| APPEND | INPUT | 10.0.0.0/24 | TCP      | 21   | DROP   | Deny the 10.0.0.0 network FTP access       |
| APPEND | INPUT | 10.0.0.0/24 | TCP      | 80   | ACCEPT | Allow the 10.0.0.0 network HTTP access     |
| APPEND | INPUT | 10.0.0.0/24 | ANY      | ANY  | DROP   | Deny all traffic from the 10.0.0.0 network |

The first step was to inspect the default firewall rules using the command:

*`sudo iptables -L`*

The output showed that all default policies (INPUT, FORWARD, OUTPUT) were set to ACCEPT, and no specific rules were applied. This means all traffic was allowed. To simulate a real-world scenario, we assumed the system hosts a web and FTP server, and has two clients: admin (10.0.0.1) and user (10.0.0.2).

```

cybrary@localhost: ~
File Edit View Search Terminal Help
cybrary@localhost:~$ sudo iptables -L
Chain INPUT (policy ACCEPT)
target prot opt source destination
Chain FORWARD (policy ACCEPT)
target prot opt source destination
Chain OUTPUT (policy ACCEPT)
target prot opt source destination
cybrary@localhost:~$ sudo iptables -A INPUT -s 10.0.0.1 -p tcp --dport 22 -j ACCEPT
cybrary@localhost:~$

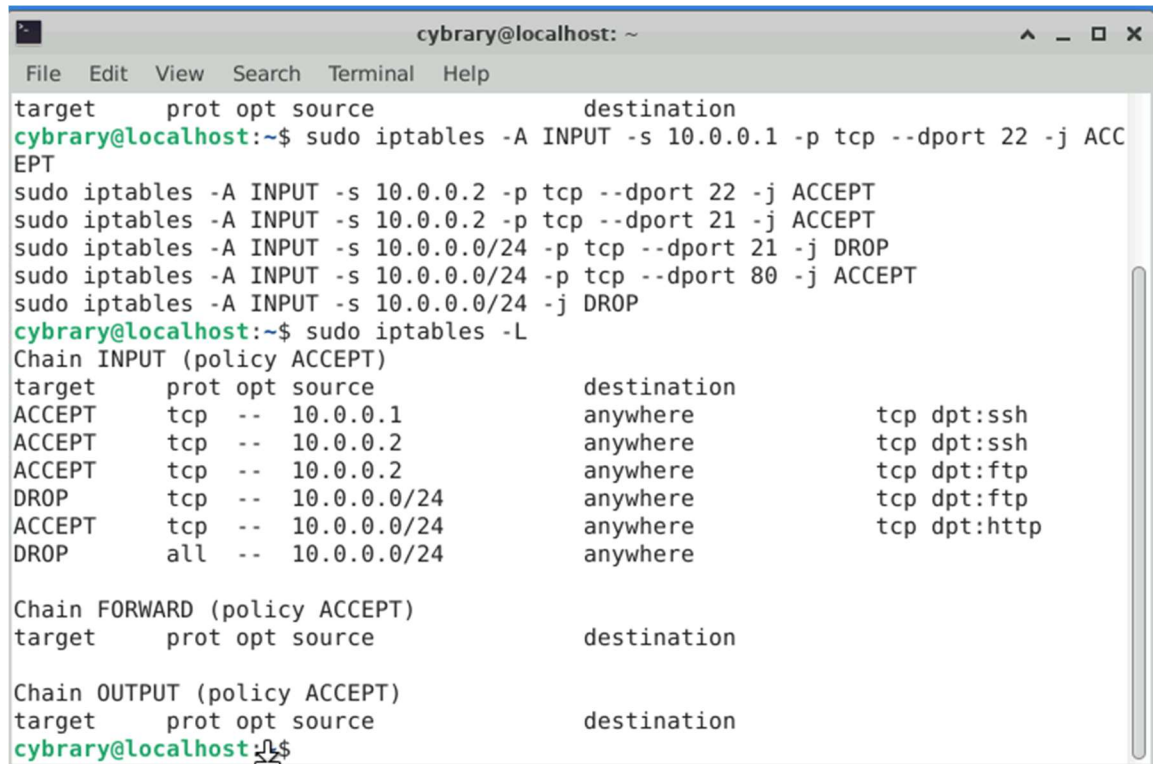
```

We added the following rules step by step:

- **Allow SSH (port 22) from admin:**  
`sudo iptables -A INPUT -s 10.0.0.1 -p tcp --dport 22 -j ACCEPT`
- **Drop SSH access from the rest of the subnet:**  
`sudo iptables -A INPUT -s 10.0.0.0/24 -p tcp --dport 22 -j DROP`
- **Allow FTP (port 21) from user only:**  
`sudo iptables -A INPUT -s 10.0.0.2 -p tcp --dport 21 -j ACCEPT`
- **Drop FTP access from others:**  
`sudo iptables -A INPUT -s 10.0.0.0/24 -p tcp --dport 21 -j DROP`
- **Allow HTTP (port 80) from entire subnet:**  
`sudo iptables -A INPUT -s 10.0.0.0/24 -p tcp --dport 80 -j ACCEPT`

- Drop all remaining traffic from 10.0.0.0/24:

```
sudo iptables -A INPUT -s 10.0.0.0/24 -j DROP
```



The screenshot shows a terminal window titled 'cybrary@localhost: ~'. The user has executed several iptables commands to configure the firewall. The output of the 'sudo iptables -L' command is displayed, showing the rules for the INPUT, FORWARD, and OUTPUT chains. The INPUT chain has a policy of ACCEPT and contains six rules. The FORWARD and OUTPUT chains also have a policy of ACCEPT and are currently empty.

```
target      prot opt source                destination
cybrary@localhost:~$ sudo iptables -A INPUT -s 10.0.0.1 -p tcp --dport 22 -j ACCEPT
Chain INPUT (policy ACCEPT)
target      prot opt source                destination
ACCEPT     tcp  --  10.0.0.1                anywhere            tcp dpt:ssh
ACCEPT     tcp  --  10.0.0.2                anywhere            tcp dpt:ssh
ACCEPT     tcp  --  10.0.0.2                anywhere            tcp dpt:ftp
DROP       tcp  --  10.0.0.0/24             anywhere            tcp dpt:ftp
ACCEPT     tcp  --  10.0.0.0/24             anywhere            tcp dpt:http
DROP       all  --  10.0.0.0/24             anywhere

Chain FORWARD (policy ACCEPT)
target      prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target      prot opt source                destination
cybrary@localhost:~$
```

### 3. Part 2 – Testing IPTables Rules

We tested the configured firewall using Mininet to simulate the two clients (admin and user). We assigned an IP address to the firewall interface:

```
sudo ip addr add 10.10.10.10/24 dev eth0
```

```
cybrary@localhost: ~
File Edit View Search Terminal Help
DROP      all  --  10.0.0.0/24      anywhere
DROP      all  --  10.0.0.0/24      anywhere
ACCEPT    tcp  --  10.0.0.0/24      anywhere      tcp dpt:http

Chain FORWARD (policy ACCEPT)
target     prot opt source      destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source      destination
cybrary@localhost:~$ sudo iptables -D INPUT 5
cybrary@localhost:~$ sudo ip addr add 10.10.10.10/24 dev eth0
cybrary@localhost:~$ xhost +
access control disabled, clients can connect from any host
cybrary@localhost:~$ xhost -
xhost: bad hostname " "
cybrary@localhost:~$ xhost+
Command 'xhost+' not found, did you mean:
  command 'xhost' from deb x11-xserver-utils (7.7+9build1)
Try: sudo apt install <deb name>
cybrary@localhost:~$ xhost +
access control disabled, clients can connect from any host
cybrary@localhost:~$ clabnet
mininet> xterm h1 h2
mininet>
```

Then we launched the simulated network using Mininet:

### Clabnet

Two xterm terminals were opened for h1 (admin) and h2 (user). Their IPs were confirmed using:

### ip a

```
"Node: h1"
t qlen 1000
  link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
  inet 127.0.0.1/8 scope host lo
    valid_lft forever preferred_lft forever
  inet6 ::1/128 scope host
    valid_lft forever preferred_lft forever
2: h1-eth0@if3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue
P group default qlen 1000
  link/ether 96:ac:f1:dd:3f:2d brd ff:ff:ff:ff:ff:ff link-netnsid 0
  inet 10.0.0.1/24 brd 10.0.0.255 scope global h1-eth0
    valid_lft forever preferred_lft forever
  inet6 fe80::94ac:f1ff:fedd:3f2d/64 scope link
    valid_lft forever preferred_lft forever
root@localhost:/home/cybrary# nmap -sT 10.10.10.10
Starting Nmap 7.80 ( https://nmap.org ) at 2025-07-16 13:44 UTC
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 3.11 seconds
root@localhost:/home/cybrary#

"Node: h2"
2: h2-eth0@if4: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue
P group default qlen 1000
  link/ether f6:35:65:d4:fa:6f brd ff:ff:ff:ff:ff:ff link-netnsid 0
  inet 10.0.0.2/24 brd 10.0.0.255 scope global h2-eth0
    valid_lft forever preferred_lft forever
  inet6 fe80::f435:65ff:fed4:fa6f/64 scope link
    valid_lft forever preferred_lft forever
root@localhost:/home/cybrary# nmap -sT 10.10.10.10
Starting Nmap 7.80 ( https://nmap.org ) at 2025-07-16 13:44 UTC
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 3.10 seconds
root@localhost:/home/cybrary#
```

Netcat was used to simulate service ports:

- sudo nc -klvp 21 (FTP)
- sudo nc -klvp 80 (HTTP)

We used Nmap from h1 and h2 to test access to the firewall IP 10.10.10.10:

nmap -sT -Pn 10.10.10.10

Expected results:

- h1 (10.0.0.1) should see ports 22 and 80 open.
- h2 (10.0.0.2) should see ports 21 and 80 open.

The screenshot displays two terminal windows. The top window, titled 'Node: h1', shows an Nmap scan of 10.10.10.10 from root@localhost. The scan reports that ports 22/tcp (ssh) and 80/tcp (http) are open. The bottom window, titled 'Node: h2', also shows an Nmap scan of 10.10.10.10 from root@localhost, reporting that ports 21/tcp (ftp) and 80/tcp (http) are open. To the right of these terminals are two Netcat listener windows. The top Netcat window shows it is listening on port 21 and has received a connection from 10.0.0.2 at 54970. The bottom Netcat window shows it is listening on port 80 and has received three connections from 10.0.0.1 (38802), 10.0.0.2 (49692), and 10.0.0.2 (49708).

```
root@localhost:/home/cybrary# nmap -sT 10.10.10.10
Starting Nmap 7.80 ( https://nmap.org ) at 2025-07-16 14:14 UTC
Nmap scan report for 10.10.10.10
Host is up (0.010s latency).
Not shown: 998 filtered ports
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http

Nmap done: 1 IP address (1 host up) scanned in 17.58 seconds
root@localhost:/home/cybrary#
```

```
cybrary@localhost:~$ sudo nc -klvp 21
Listening on 0.0.0.0 21
Connection received on 10.0.0.2 54970
```

```
root@localhost:/home/cybrary# nmap -sT 10.10.10.10
Starting Nmap 7.80 ( https://nmap.org ) at 2025-07-16 14:14 UTC
Nmap scan report for 10.10.10.10
Host is up (0.0027s latency).
Not shown: 998 filtered ports
PORT      STATE SERVICE
21/tcp    open  ftp
80/tcp    open  http

Nmap done: 1 IP address (1 host up) scanned in 17.98 seconds
root@localhost:/home/cybrary#
```

```
cybrary@localhost:~$ sudo nc -klvp 80
Listening on 0.0.0.0 80
Connection received on 10.0.0.1 38802
Connection received on 10.0.0.2 49692
Connection received on 10.0.0.2 49708
```

## 4. Observations and Challenges

- Rule order in iptables is crucial. If DROP comes before ACCEPT, the traffic will be denied.
- After adding a new rule, it must be placed correctly using iptables -I or by rewriting rules.
- A final DROP ALL rule can eliminate the need for individual DROP rules.

## 5. Conclusion

This lab provided hands-on experience with iptables and gave me the confidence to understand how firewalls behave in real environments. I practiced rule creation, rule ordering, testing with Nmap, and used simulated hosts and services. This practical understanding supports my goal to work in a SOC environment where analyzing and managing traffic rules is a daily responsibility.